

```

1  import java.io.*;
2  import java.net.*;
3
4  public class Comanche implements Runnable {
5      private Socket s;
6
7      public Comanche(Socket s) {
8          this.s = s;
9      }
10
11     public void run() {
12         try {
13             InputStreamReader in = new InputStreamReader(s.getInputStream());
14             // Stream qu'on peut utiliser pour écrire comme avec System.out.println
15             PrintStream out = new PrintStream(s.getOutputStream());
16             // Reader capable de retenir les numéros de ligne
17             String rq = new LineNumberReader(in).readLine();
18             System.out.println(rq);
19             if (rq.startsWith("GET ")) {
20                 File f = new File(rq.substring(5, rq.indexOf(' ', 4)));
21                 if (f.exists() && !f.isDirectory()) {
22                     InputStream is = new FileInputStream(f);
23                     byte[] data = new byte[is.available()];
24                     is.read(data);
25                     is.close();
26                     String s = new String(data);
27                     out.print("HTTP/1.0 200 OK\n\n" + s);
28                 } else {
29                     out.print("HTTP/1.0 404 Not Found\n\n <html>Document not found.</html>");
30                 }
31             }
32             out.close();
33             s.close();
34         } catch (IOException ex) {
35             ex.printStackTrace();
36         }
37     }
38
39     public static void main(String[] args) throws IOException {
40         ServerSocket s = new ServerSocket(Integer.parseInt(args[0]));
41         System.out.println(
42             "Comanche is running. Open http://localhost:"
43             + args[0]
44             + "</any file in current directory> in your web browser.");
45         System.out.println(
46             "In the terminal, you can also run: curl localhost:"
47             + args[0]
48             + "</any file in current directory>");
49         while (true) {
50             new Thread(new Comanche(s.accept())).start();
51         }
52     }
53 }
54
55 import java.io.IOException;

```

```

56 import java.io.InputStream;
57 import java.io.OutputStream;
58 import java.net.ServerSocket;
59 import java.net.Socket;
60 import java.util.Random;
61
62 public class LoadBalancer extends Thread {
63     static String hosts[] = {"localhost", "localhost"};
64     static int ports[] = {8081, 8082};
65     static int nbHosts = 2;
66     static Random rand = new Random();
67     Socket cli;
68
69     public LoadBalancer (Socket s) {
70         cli = s;
71     }
72
73     public static void main(String args[]) {
74         try {
75             ServerSocket ss = new ServerSocket(8080);
76             while (true) {
77                 Socket s = ss.accept();
78                 new LoadBalancer(s).start();
79             }
80         } catch (IOException e) {
81             e.printStackTrace();
82         }
83     }
84
85     public void run() {
86         try {
87             int nServ = rand.nextInt(nbHosts);
88             Socket serv = new Socket(hosts[nServ], ports[nServ]);
89             InputStream cliIS = cli.getInputStream();
90             OutputStream cliOS = cli.getOutputStream();
91             InputStream servIS = serv.getInputStream();
92             OutputStream servOS = serv.getOutputStream();
93             byte buff[] = new byte[1024];
94             int nb = cliIS.read(buff, 0, 1024);
95             servOS.write(buff, 0, nb);
96             nb = servIS.read(buff, 0, 1024);
97             cliOS.write(buff, 0, nb);
98             cli.close();
99             serv.close();
100         } catch (IOException e) {
101             e.printStackTrace();
102         }
103     }
104 }

```